

Python Code

```
1  """
2  Code to implement the game of Spot it!
3
4  http://www.blueorangegames.com/spotit/
5
6  For each function, replace the return statement with your code. Add
7  whatever helper functions you deem necessary.
8  """
9
10 import comp140_module2 as spotit
11
12 def equivalent(point1, point2, mod):
13     """
14     Determines if the two given points are equivalent in the projective
15     geometric space in the finite field with the given modulus.
16
17     Each input point, point1 and point2, must be valid within the
18     finite field with the given modulus.
19
20     inputs:
21         - point1: a tuple of 3 integers representing the first point
22         - point2: a tuple of 3 integers representing the second point
23         - mod: an integer representing the modulus
24
25     returns: a boolean indicating whether or not the points are equivalent
26     """
27
28     # use cross product as described in assignment description
29     return (point1[0] * point2[2] % mod == point1[2] * point2[0] % mod and
30             point1[0] * point2[1] % mod == point1[1] * point2[0] % mod
31             and point1[1] * point2[2] % mod == point1[2] * point2[1] % mod)
32
33 def incident(point, line, mod):
34     """
35     Determines if a point lies on a line in the projective
36     geometric space in the finite field with the given modulus.
37
38     The inputs point and line must be valid within the finite field
39     with the given modulus.
40
41     inputs:
42         - point: a tuple of 3 integers representing a point
43         - line: a tuple of 3 integers representing a line
44         - mod: an integer representing the modulus
45
46     returns: a boolean indicating whether or not the point lies on the line
47     """
48     # we check the fourth property of projective geometry
49     return (point[0] * line[0] + point[1] * line[1] + point[2] * line[2]) % mod == 0
50
```

```

51
52 def generate_all_points(mod):
53     """
54     Generate all unique points in the projective geometric space in
55     the finite field with the given modulus.
56
57     inputs:
58         - mod: an integer representing the modulus
59
60     Returns: a list of unique points, each is a tuple of 3 elements
61     """
62     points = []
63
64     # Logic explained. Total number of points is mod^3. 0,0,0 not allowed.
65     # Each point(a,b,c) corresponds to 1 * (a,b,c), ... ,mod-1 * (a,b,c) vectors
66     # therefore, total points is (mod^3 - 1)/(mod - 1) = p^2+p+1
67
68     # first type of points, first entry is 1, the rest are between 0 and mod
69     for first in range(mod):
70         for second in range(mod):
71             points.append((1, first, second))
72
73     # second type of points, first entry is 0, second is 1, the third is between 0 and
mod
74     for third in range(mod):
75         points.append((0, 1, third))
76
77     # special case when first two are 0
78     points.append((0, 0, 1))
79
80     return points
81
82 def create_cards(points, lines, mod):
83     """
84     Create a list of unique cards.
85
86     Each point and line within the inputs, points and lines, must be
87     valid within the finite field with the given modulus.
88
89     inputs:
90         - points: a list of unique points, each represented as a tuple of 3 integers
91         - lines: a list of unique lines, each represented as a tuple of 3 integers
92         - mod: an integer representing the modulus
93
94     returns: a list of lists of integers, where each nested list represents a card.
95     """
96     # list of cards
97     cards = []
98
99     # go over all the lines
100    for line in lines:
101        card = []
102        # check all points
103        for idx, point in enumerate(points):

```

```
104         # if that point is on the line we want, add its index to the card
105         if incident(point, line, mod):
106             card.append(idx)
107         cards.append(card)
108
109     return cards
110
111 def run():
112     """
113     Create the deck and play the game.
114     """
115     # Prime modulus
116     # Set to 2 or 3 during development
117     # Set to 7 for the actual game
118     modulus = 7
119
120     # Generate all unique points for the given modulus
121     points = generate_all_points(modulus)
122
123     # Lines are the same as points, so make a copy
124     lines = points[:]
125
126     # Generate a deck of cards given the points and lines
127     deck = create_cards(points, lines, modulus)
128
129     # Run GUI - uncomment the line below after you have implemented
130     #             everything and you can play your game. The GUI does
131     #             not work if the modulus is larger than 7.
132
133     spotit.start(deck)
134
135     # Uncomment the following line to run your game (once you have
136     # implemented the run function.)
137
138     run()
139
```